Eric Romero
CSC 445
4/20/2026

Homework 10

Algorithm Description:

The point of this algorithm is to create a graph of a given maze and use breadth-first search to find the shortest from top left to bottom right corners. Each cell has a number that represents a jump distance in a direction.

To proceed, BFS is used because it will find all possible positions that are reachable in a few moves followed by moves that may take more. Thus, leads to a path found with a minimal length.

We use a queue of Position objects in order to proceed with BFS. It will hold cells that have been found. We will also use a 2D array of strings that marks visited cells and records the path to reach them.

As stated previously, we start at the top left corner, position (0, 0) (meaning first element of rows and columns) and try to reach the goal (n - 1, n - 1) (meaning the last element of rows and columns). It will take the next cell in line within the queue, check if it is the bottom left, and if not, proceed to jump determined by the jump value in the position. If the move is legal, it goes to that corresponding cell, added to the queue, and records the path into the array.

Ultimately, the path is found incrementally with the use of BFS. With each new cell that is processed, the path is recorded by appending the directional move used to reach these cells. If no legal path is found, an empty string is returned, otherwise a string with the directions of the shortest legal path is returned.

Eric Romero
CSC 445
4/20/2026

Time Complexity

Let n be the length of the maze → n^2 positions possible

With BFS:

each position is visited once and checks 4 possible directions/moves -> O(n^2)

With Path Array:

New string created by copying the old one and adding a new character corresponding to the direction BFS proceeds to go in -> O(n^2)

Worst case time complexity:

O(n^4)

Space Complexity

Queue: holds up to O(n^2) positions
Path array (size n x n): Holds up to O(n^2) references

Total string storage would be O(n^2) x O(n^2) = O(n^4)

Pseudocode

```
solveMaze(maze):
    N = maze.length

    Create Queue, Position objects, ‘q’
    Create 2d array, size n x n, ‘path’

    q.add(Position(0,0))
    Path[0][0] = “”

    While q is not empty:
        Cur = q.poll()
        I = cur.row
        J = cur.col

        If i == n - 1 and j == n - 1:
            Return path[i][j]

        V = maze[i][j]

        // up
        If i - v >= 0 and path[i - v][j] == null:
```

Eric Romero
CSC 445
4/20/2026

```
                q.add(Position(i - v, j)
                Path[i - v][j] = path[i][j] + “U”

        // down
        If i + v < n and path[i + v][j] == null:
                q.add(Position(i + v, j)
                Path[i + v][j] = path[i][j] + “D”
        // left
        If j - v >= 0 and path[i][j - v] == null:
                q.add(Position(i, j - v)
                Path[i][j - v] = path[i][j] + “L”
        // right
        If j + v < n and path[i][j + v] == null:
                q.add(Position(i, j + v)
                Path[i][j + v] = path[i][j] + “R”

    Return “”
```

Pseudocode Review:

Variables/Data structures

- Int n: Stores maze length
- Queue<position> q: For BFS. Stores discovered positions.
- String[n][n] Path: Tracks visited positions and path to reach current position.
    - If == null: position not yet visited
    - If != null: position already visited

- Position cur: Current position taken from queue to process.
- Int i, j: Row and column of current position.
- Int v: Jump distance

Control Blocks

- While (!q.isEmpty): Repeats BFS process until final position is found or if ultimately unreachable.
- If (i == n - 1 && j == n - 1): Checks the current position and whether it is the ending point. Should return the path stores in ‘path’
- Direction Checks:
    - The jump off point from the current position is valid within the bounds of the maze
    - The position it reaches hasn’t been seen yet
    - If both are true, that new position is added to the queue and the BFS process continues
- Final return: Empty string. Means that we do not reach the goal, meaning that there is no legal path that exists.

Eric Romero
CSC 445
4/20/2026

Correctness:

1. Show that the algorithm returns a valid path
2. Show the path returned has the fewest number of legal moves.

Invariant: When path[i][j] is not null, we store a path from position (0, 0) to (i, j).

Initially, this is true. Path[0][0] is an empty string, meaning correct start position. During search, we check the position at (i,j) where it will create a new path if the move is legal. This new path is recorded by adding the direction to the path string stored for the position. Through induction, every not-null entry in the path string is a valid path from the start to that current position.

The path string is returned only when the goal position is reached (n - 1, n- 1), and removed from the queue. By the invariant, the path string is valid from the start to the goal. If the queue is empty and the algorithm returns an empty string, then that means that all possible positions to explore have been explored, but the goal is never reached, so no solution is possible.

To prove the path is the shortest, consider BFS explores positions in an increasing order of possible moves from the starting position. Following a base, once the first cell is found, it is then presumed to be the shortest path. Because ‘path’ is only assigned once to that position, the stored cumulative path is the shortest – This is all pretty redundant to be honest – but by this logic, the path that is created for the end position should be the shortest legal valid path since the beginning position.